An Application Development Project Report
Project Title: NEUROFLIGHT EMG – Controlled Drone Navigation
Designed & Developed By:

2411CS050003 – A. Revanth
2411CS050017 – D. Abhiram
2411CS050024 – G. Nandeeshwar
2411CS050030 – D. Vinay
2411CS050032 – J. Mani Teja
2411CS050043 – M. Akshath
2411CS050047 – M. Anil
2411CS050051 – N. Sruthi
2411CS050082 – B. Jatin
2411CS050089 – U. Pavithra

Guided By:
Dr. B. Nageshwar Rao (Assistant Professor)
Department: Computer Science & Engineering (Internet of Things)
Year/Semester: 3YR-ISEM
University: Malla Reddy University, Hyderabad

Date: November 2026

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science & Engineering (Internet of Things)

CERTIFICATE

This is to certify that this is the bonafide record of the application development entitled **"NEUROFLIGHT – EMG–Controlled Drone Navigation"** submitted by the below students of B. Tech III year I semester, Department of CSE (IoT) during the year 2026-27. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.

2411CS050003 - A.Revanth
2411CS050017 - D.Abhi Ram
2411CS050024 - G.Nandeeshwar
2411CS050030 - D.Vinay
2411CS050032 - Mani Teja
2411CS050043 - M.Akshath
2411CS050047 - M.Anil
2411CS050051 - N.Sruthi
2411CS050082 - B.Jatin
2411CS050089 - U.Pavithra

Internal Guide
Mrs. T. Suneetha
(Assistant Professor)

Dean (CS, IOT&ECE)
Dr G. Anand Kumar
Professor & Dean

External Examiner

DECLARATION

We hereby declare that the project report entitled “NEUROFLIGHT” has been carried out by us. This work has been submitted to the Department of Computer Science and Engineering (Internet of Things), Malla Reddy University, Hyderabad in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology. we further declare that this project work has not been submitted in full or part for the award of any other degree in other educational institutions.

Place:

Date:

2411CS050003 - A.Revanth
2411CS050017 - D.Abhi Ram
2411CS050024 - G.Nandeeshwar
2411CS050030 - D.Vinay
2411CS050032 - Mani Teja
2411CS050043 - M.Akshath
2411CS050047 - M.Anil
2411CS050051 - N.Sruthi
2411CS050082 - B.Jatin
2411CS050089 - U.Pavithra

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude **to Dr. V. S. K Reddy, Vice Chancellor,** for his visionary leadership and unwavering commitment to academic excellence.

We would also like to express my deepest appreciation to our project guide **Dr. B. Nageshwar Rao, Assistant Professor** whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We extend our gratitude to our **PRC – convenor, Dr. G. Latha, Associate professor,** for giving valuable inputs and timely guidelines to improve the quality of our project through a critical review process. We thank our project coordinator **Dr. B. Nageshwar Rao,** for his timely support.

We are also grateful **to Dr. G. Anand Kumar, Dean C**, for providing us with the necessary resources and facilities to carry out this project.

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

2411CS050003 - A.Revanth
2411CS050017 - D.Abhi Ram
2411CS050024 - G.Nandeeshwar
2411CS050030 - D.Vinay
2411CS050032 - Mani Teja
2411CS050043 - M.Akshath
2411CS050047 - M.Anil
2411CS050051 - N.Sruthi
2411CS050082 - B.Jatin
2411CS050089 - U.Pavithra

ABSTRACT

NeuroFlight is a project that aims to control a drone using human muscle signals. It uses a technology called Electromyography (EMG) to detect the electrical signals produced by muscles when a person moves or contracts them. These signals are collected using EMG sensors placed on the user's arm or hand. The main aim of this project is to make drone control easier and more natural without using a traditional remote controller.

In this project, EMG sensors collect muscle signals from the user's hand movements. These signals are sent to a microcontroller, where they are processed and converted into commands. Based on the detected signals, the drone receives instructions to move in different directions, such as forward, backward, left, and right. This helps the user control the drone through simple muscle movements.

The system uses wireless communication to send the processed commands from the controller to the drone. The drone receives these commands and performs the required movements. This method reduces the need for manual control and provides a new way of interacting with drones. It also helps in understanding how human muscle signals can be used in modern technology.

The main objective of NeuroFlight is to develop a simple, user-friendly, and efficient drone control system using EMG technology. This project can be useful in areas such as assistive technology, research, and human-machine interaction. Overall, NeuroFlight combines human muscle signals with drone technology to create an innovative and easy method of drone navigation.

Keywords: *NeuroFlight, Electromyography (EMG), EMG Signal Processing, Muscle Signal Detection, Human-Machine Interaction, Gesture-Based Control, Drone Navigation, Wireless Communication, ESP32-S3, LiteWing Drone, Signal Classification, Intelligent Drone Control, Assistive Technology.t*

Chapter	Title	Page No.
Chapter 1	**Introduction**	**1 – 3**
1.1	Problem Definition & Description	2
1.2	Objective of the Project	2 – 3
1.3	Scope of the Project	3
Chapter 2	**System Analysis**	**4 – 13**
2.1	Existing System	4 – 5
2.1.1	Background & Literature Survey	4 – 5
2.1.2	Limitations of Existing System	5
2.2	Proposed System	6
2.2.1	Advantages of Proposed System	6
2.3	Software & Hardware Requirements	7
2.3.1	Software Requirements	7
2.3.2	Hardware Requirements	7
2.4	Feasibility Study	8 – 9
2.4.1	Technical Feasibility	8
2.4.2	Robustness & Reliability	8
2.4.3	Economic Feasibility	9
Chapter 3	**Architectural Design**	**10 – 17**
3.1	Modules Design	10
3.2	Method and Algorithm Design	11
3.3	Project Architecture	12 – 17
3.3.1	Architectural Diagram	12
3.3.2	Data Flow Diagram	13
3.3.3	Class Diagram	14
3.3.4	Use Case Diagram	15
3.3.5	Sequence Diagram	16
3.3.6	Activity Diagram	17

Chapter	Title	Page No.
Chapter 4	**Implementation & Testing**	**17 – 21**
4.1	Coding Blocks	17 – 20
4.2	Test Cases	20 – 21
4.2.1	Test Case 1	20
4.2.2	Test Case 2	20
4.2.3	Test Case 3	21
4.2.4	Test Case 4	21
Chapter 5	**Results**	**22 – 30**
5.1	Resulting Screens	22 – 26
5.1.1	Sign In Page	22
5.1.2	Report Issue	23
5.1.3	Track a Complaint	24
5.1.4	Admin Access	25
5.1.5	Admin Panel	25
5.1.6	Field Officer Login	26
5.1.7	Field Officer Panel	26
5.2	Resulting Table	27
5.2.1	System Performance Metrics	27
5.2.2	Existing System vs MARTS	27
5.3	Resulting Graph	28 – 29
5.3.1	Area Wise Complaint Distribution	28
5.3.2	Incident Response Resolution Timeline	28
5.3.3	Complaint Priority Distribution	29
5.3.4	Complaint Status Distribution Graph	29
5.4	Resulting Analysis	30
5.4.1	Time Complexity	30
5.4.2	Space Complexity	30

Chapter	Title	Page No.
5.4.3	Results Summary	30
Chapter 6	**Conclusion & Future Scope**	**31**
6.1	Conclusions	31
6.2	Future Scope	31
	References	32
	Poster Presentation	33

Section	Name	Page No.
List Of Diagrams :		
3.3.1	Architectural Diagram	12
3.3.2	Data Flow Diagram	13
3.3.3	Class Diagram	14
3.3.4	Use Case Diagram	15
3.3.5	Sequence Diagram	16
3.3.6	Activity Diagram	17
List Of Screen Shots		
5.1.1	Sign In Page	22
5.1.2	Report Issue	23
5.1.3	Track a Complaint	24
5.1.4	Admin Access	25
5.1.5	Admin Panel	25
5.1.6	Field Officer Login	26
5.1.7	Field Officer Panel	26
List Of Resultant Table		
5.2.1	System Performance Metrics	27
5.2.2	Existing System vs MARTS	27

CHAPTER 1

INTRODUCTION

NeuroFlight: EMG-Controlled Drone Navigation is an innovative project that allows a person to control a drone using human muscle movements instead of a traditional remote controller. In normal drone systems, users operate the drone using joysticks, buttons, or mobile applications. In this project, muscle signals are used as input commands to control the drone. The main idea is to create a simple and user-friendly system that connects human muscle activity with drone technology.

EMG stands for Electromyography, a technique used to detect the electrical signals produced by muscles when they contract. In this project, an EMG sensor is placed on the user's arm or forearm to collect these signals. When the user moves or tightens their muscles, the electrical activity changes, and the sensor detects these changes. The collected signals are then sent to a controller for processing. This helps the system recognize different muscle movements and use them as inputs for drone control.

The working principle of NeuroFlight involves collecting, processing, and converting muscle signals into drone commands. First, the EMG sensor detects electrical activity from the user's muscles. Next, a microcontroller receives the signals and processes them according to predefined conditions. Based on the detected muscle activity, the controller generates commands for specific drone movements. These commands are transmitted to the drone through a suitable communication system, allowing it to respond to the user's muscle movements.

The system can be programmed to control different drone movements, such as moving forward, backward, left, right, ascending, and descending. Each movement can be assigned to a specific muscle gesture or signal range. When the user performs the required muscle movement, the system identifies the signal and sends the corresponding command to the drone. The accuracy of these movements depends on proper sensor placement, signal processing, calibration, and the control logic used in the project.

NeuroFlight has potential applications in robotics, education, research, surveillance, and human-machine interaction. It provides an alternative way to control drones and helps demonstrate how biological signals can be used in electronic systems. This technology may also support the development of assistive control systems for people who find conventional controllers difficult to use. By combining EMG sensors, microcontrollers, and drone technology, the project provides practical knowledge of biomedical sensors, embedded systems, and wireless communication.

In conclusion, NeuroFlight provides an innovative way to control a drone using human muscle signals instead of a traditional remote controller. In the future, this system can be improved to achieve better accuracy, faster response, and safer drone navigation.

1.1 Problem Definition & Description

Traditional drone systems are mainly controlled using remote controllers, joysticks, or mobile applications. These methods require users to operate physical buttons and controls to move the drone in different directions. For some users and certain applications, using these conventional control methods can be difficult or inconvenient. Therefore, there is a need for an alternative control system that allows users to operate a drone through simple human movements.

NeuroFlight aims to solve this problem by using EMG (Electromyography) sensors to detect electrical signals produced by human muscles. These signals change when a person contracts or relaxes their muscles. By detecting these changes, the system can use muscle activity as an input for drone control. This provides an alternative method of controlling a drone without depending entirely on traditional remote-control devices.

The proposed system consists of an EMG sensor, a microcontroller, a communication module, and a drone. The EMG sensor collects muscle signals from the user's arm or forearm and sends them to the microcontroller. The microcontroller processes the signals and identifies the corresponding commands based on predefined conditions. These commands are then transmitted to the drone through a suitable communication system to control its movement.

One of the main challenges in developing an EMG-controlled drone is accurately detecting and interpreting muscle signals. These signals may vary depending on muscle strength, sensor placement, and the user's movements. Unwanted noise or incorrect signal detection can result in unexpected drone movements. Therefore, proper sensor placement, signal filtering, calibration, and reliable command processing are important for improving the accuracy and stability of the system.

The main objective of NeuroFlight is to develop a simple and user-friendly drone control system using human muscle signals. The EMG sensor detects muscle activity and sends signals to the controller for processing. These signals are converted into commands to control the drone's movement. This project provides an alternative to traditional drone controllers and aims to improve control flexibility and user interaction.

1.2 Objectives of the Project

The main objective of NeuroFlight is to develop an innovative drone control system using human muscle signals. It aims to control a drone without depending on a traditional remote controller. The system uses an EMG sensor to detect electrical signals produced by the user's muscles. These signals are processed and converted into commands for drone navigation. The project aims to make drone control simple, efficient, and user-friendly.

The project uses an EMG sensor to detect electrical signals from the user's muscles. These signals are sent to a microcontroller for processing.

The controller identifies the muscle movements based on predefined conditions. It converts the detected signals into commands for the drone. These commands help the drone move in different directions according to the user's muscle movements.

The project aims to explore the use of biomedical sensors in drone technology. It combines EMG sensors, microcontrollers, and wireless communication for drone navigation. It also focuses on improving control accuracy and system reliability. In the future, it can support the development of hands-free and assistive control systems.

1.3 Scope of the Project

The scope of NeuroFlight is to develop a drone control system using human muscle signals. The system uses an EMG sensor to detect electrical activity from the user's muscles. These signals are processed by a microcontroller. The processed signals are converted into commands for drone navigation. This provides an alternative to traditional drone control methods.

The scope of NeuroFlight is to develop a drone control system using human muscle signals. The system uses an EMG sensor to detect electrical activity from the user's muscles. These signals are processed by a microcontroller. The processed signals are converted into commands for drone navigation. This provides an alternative to traditional drone control methods.

In the future, the system can be improved by using advanced signal processing techniques. Better muscle movement recognition can help improve control accuracy. Additional safety features can help prevent unexpected drone movements. More drone functions can be added to improve navigation. These improvements may support hands-free control and assistive technologies.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Existing System

The existing drone control systems mainly use remote controllers to operate drones. These controllers contain joysticks and buttons to control the direction and movement of the drone. Users can move the drone forward, backward, left, and right using these controls. This method is widely used because it is simple and provides direct control over drone movements.

Another existing method is controlling drones through mobile applications. In this system, users use a smartphone screen to send commands to the drone. The application provides virtual buttons or touch controls to manage different drone movements. However, this method depends on the mobile device and may require users to continuously interact with the screen.

Some advanced drone systems use GPS and sensors for automatic navigation and stability. These technologies are useful for photography, mapping, and surveillance. However, they do not directly use human muscle signals to control drone movements.

Existing drone control systems mainly depend on remote controllers, mobile applications, or automated commands. Users need to operate physical buttons or touchscreens to control the drone. These systems do not directly use human muscle signals as input. Therefore, EMG-based control offers an alternative approach to drone navigation.

2.1.1 Background & Literature review

Drone technology is widely used in photography, agriculture, surveillance, and research. Traditional drones are controlled using remote controllers, joysticks, or mobile applications. Recent research explores alternative control methods using human gestures and muscle signals. EMG sensors detect electrical signals produced by muscles during contraction. These signals can be processed and converted into commands for controlling electronic devices. The NeuroFlight project uses this technology to explore muscle-based drone navigation. This approach offers an alternative to traditional drone control methods.

Literature Survey

- **EMG-Based Control of a Quadcopter Management**

Authors: Maham Ali, Areeba Riaz.

Publication: MECnIT 2020 Conference.

This study presents a method for controlling a quadcopter using EMG signals collected from the user's arm muscles. The researchers used muscle sensors to detect four different hand gestures and convert them into commands for drone movement. The signals were filtered to remove noise, and a

classification algorithm was used to identify the gestures. This research is closely related to NeuroFlight because it demonstrates how muscle signals can be processed to control a drone in different directions

- **EMG-Based Drone Control**

Authors: S. Longhi, D. Marzioni

Journal: M. McCoggle et al**.**

This study uses EMG sensors and an Artificial Neural Network (ANN) to control drones through arm movements .Muscle signals are processed and converted into commands for drone navigation. This research supports the use of EMG technology in the NeuroFlight project.

- **Data-Driven Body–Machine Interface for Drone**

Authors: Jenifer Miehlbradt, Alexandre Cherpillod

Journal: Proceedings of the National Academy of Sciences (PNAS)

This research explores controlling drones through natural body movements and muscle activity. It aims to provide an easy and user-friendly alternative to traditional joystick control. The experiments showed that this method could help users learn drone navigation more quickly. This study supports the idea of using human muscle signals for NeuroFlight drone control.

2.1.2 Limitations of Existing System

The existing drone control systems mainly depend on remote controllers, joysticks, and mobile applications. These methods require users to operate physical buttons or touchscreens continuously, which may be inconvenient in certain situations. Traditional drone systems may require practice and training to achieve accurate control. Beginners can find it difficult to coordinate different controls, and incorrect inputs may cause unwanted drone movements.

Most existing systems do not directly use human muscle signals to control drone navigation. They depend on manual inputs or predefined commands, limiting the use of muscle-based interaction. quality of service.

Another limitation is that conventional controllers may not be suitable for every user, especially those who have difficulty operating physical controls. This creates a need to explore alternative control methods that offer greater flexibility.

Existing systems also have limited direct interaction between human muscle activity and drone movement. Therefore, EMG-based technology can be explored as an alternative approach to make drone control more interactive and accessible.

2.2 Proposed System

The proposed NeuroFlight system is designed to control a drone using human muscle signals instead of a traditional remote controller. It uses an EMG (Electromyography) sensor to detect electrical signals produced by the user's muscles. The system provides an alternative way to control drone movements through simple muscle actions. Its main aim is to make drone navigation more interactive and user-friendly.

The EMG sensor is placed on the user's arm or forearm to collect muscle activity signals. These signals are sent to a microcontroller, which processes them and identifies the corresponding commands. Based on the programmed conditions, different muscle movements can be assigned to specific drone actions. This allows the system to convert human muscle activity into useful control instructions.

After processing the signals, the microcontroller sends the control commands to the drone through a suitable communication system. The drone responds to these commands by performing movements such as moving forward, backward, left, right, ascending, or descending, depending on the system configuration. Proper calibration and signal filtering help improve command accuracy and reduce unwanted movements.

The proposed system combines EMG sensing, embedded systems, and wireless communication to provide an alternative drone control method. It can help students and researchers understand the practical use of biomedical sensors in robotics and navigation. In the future, the system can be improved by adding better signal recognition, faster response, and emergency-stop features to enhance reliability and safety.

2.2.1 Advantages of the Proposed System

- **Easy Drone Control**
  The Neuro Flight system allows users to control a drone using muscle movements instead of a traditional remote controller. This provides an alternative and interactive method of drone navigation.

- **Real-Time Response**
  The EMG sensor detects muscle activity, and the microcontroller processes the signals to generate control commands. With proper calibration and processing, the system can respond to muscle movements with minimal delay.

- **Flexible Navigation**
  Different muscle signals can be assigned to different drone movements, such as forward, backward, left, and right.Additional commands can be included by modifying the control program.

- **Future Improvement**
  The proposed system can be enhanced with better signal filtering, accurate muscle movement recognition, and emergency-stop features.

2.3 Software & Hardware Requirements

2.3.1 Software Requirements

1. **Arduino IDE:** Used to write, compile, and upload code to the microcontroller.

2. Embedded C/C++: Used to program the controller to process EMG signals and generate drone commands.

3. ESP32 Board Package: Helps configure and program the ESP32 microcontroller using Arduino IDE.

4. Bluetooth Communication Libraries: Used to establish wireless communication between compatible devices.

5. Web Browser: Used to access the web-based interface for controlling the drone, if included in the project.

6. Required Libraries: Used to support sensor readings, communication, and other control functions.

2.3.2 Hardware Requirements:

- EMG Sensor
- ESP32 Microcontroller
- Drone / Quadcopter
- Drone Flight Controller
- Motor Drivers / ESCs
- Brushless DC Motors
- Propellers
- Li-Po Battery
- Voltage Regulator
- Jumper Wires
- Breadboard or PCB
- USB Cable

2.4 Feasibility Study

2.4.1 Technical Feasibility

The NeuroFlight project is technically feasible because it uses commonly available electronic components and sensor technology. An EMG sensor is used to detect electrical signals produced by human muscles. An ESP32 microcontroller can process these signals and generate control commands. These components can be integrated using suitable programming techniques. The system can be developed using Arduino IDE and C/C++ programming. Therefore, the project is suitable for implementation using available technology.

The system combines EMG sensing, signal processing, and wireless communication to control drone movements. The EMG sensor collects muscle signals and sends them to the microcontroller for processing. Based on predefined conditions, the controller generates commands for the drone. These commands are transmitted through a compatible communication system. Proper calibration and signal filtering can improve the accuracy of the system. This makes the proposed system technically achievable with careful development.

The project can be tested and improved using suitable hardware and software tools. The EMG sensor must be properly positioned to obtain reliable muscle signals. The controller program should be designed to recognize different muscle activities accurately. Communication between the controller and drone must be stable for effective navigation. Safety features, such as an emergency stop, should also be included. Thus, the system is technically feasible with proper testing and integration.

2.4.2 Robustness & Reliability

The NeuroFlight system is designed to provide reliable drone control by detecting muscle signals through an EMG sensor. The microcontroller processes these signals and converts them into suitable commands for drone movement. Proper sensor placement, signal filtering, and calibration help reduce errors and improve system performance. These methods help the system respond more accurately to different muscle movements.

The reliability of the system depends on accurate signal detection, stable wireless communication, and proper integration of hardware and software components. Regular testing helps identify errors and improve overall performance. Safety features, such as an emergency stop and signal-loss handling, can help prevent unexpected drone movements. These improvements can make the NeuroFlight system more stable, dependable, and suitable for practical testing.

The system can be improved through continuous testing and proper maintenance of its components. Reliable power supply and stable communication help ensure smooth drone operation. These measures improve the overall robustness and reliability of NeuroFlight.

2.4.3 Economic Feasibility

The NeuroFlight project is economically feasible because it can be developed using commonly available electronic components. The main components include an EMG sensor, ESP32 microcontroller, drone, battery, and connecting wires. These components are available in electronics stores and online marketplaces. Open-source software such as Arduino IDE can be used without purchasing a software license. This helps reduce the overall development cost of the project.

The project requires a reasonable initial investment for purchasing the necessary hardware components. The total cost depends on the type of EMG sensor, drone, microcontroller, and other accessories selected. Using reusable components and open-source development tools can help reduce expenses. The system can also be developed and tested in stages to manage the budget effectively. Therefore, the project can be suitable for students and academic institutions with limited budgets.

In the long term, the NeuroFlight system can be modified and upgraded without replacing the entire setup. Existing components can be reused for testing different programs and improving control functions. Maintenance costs can be reduced through proper handling and regular inspection of the hardware. Future improvements can be added according to the available budget and project requirements. Thus, the project offers a practical and cost-effective approach to exploring EMG-based drone navigation.

CHAPTER 3

ARCHITECTURAL DESIGN

3.1 Modules Design

The NeuroFlight system consists of different modules that work together to control the drone using human muscle signals. The EMG sensor collects electrical signals from the user's muscles, and the ESP32 processes these signals into control commands. The wireless communication module transmits the commands to the drone. Finally, the drone control module performs the required movements based on the received commands.

3.1.1 Admin Module

The Admin Module manages the settings and overall operation of the NeuroFlight system. It helps configure the EMG sensor and check connections between the system components. It allows adjustment of signal thresholds for accurate muscle signal detection. It also supports system testing and error identification for reliable drone control.

3.1.2 User Module

This module enables citizens to interact with the system. Users can register, log in, and submit complaints related to civic issues such as waste management, road repairs, or streetlights. They can also view the status of their previous complaints, making the system transparent and user-friendly.

3.1.3 Complaint Module

The Complaint Module allows users to report problems encountered while operating the NeuroFlight system. It helps identify issues related to EMG signal detection, wireless communication, or drone movement. This module supports troubleshooting and improves the overall system performance.

3.1.4 Tracking Module

The Tracking Module helps monitor the drone's movement and current operating status during navigation. It allows users to observe the drone's direction and movement based on the control commands received. This module helps improve navigation monitoring and supports safer drone operation.

3.1.5 Feedback Module

It helps monitor the detected EMG signals, classified control commands, and communication status between the user-side controller and the drone.The module helps verify whether the generated commands, such as LEFT, RIGHT, and STOP, are processed and transmitted correctly to the drone.

3.2 Method & Algorithm Design

3.2.1 Command Assignment Method

- The module helps verify whether the generated commands, such as LEFT, RIGHT, and STOP, are processed and transmitted correctly to the drone. It also supports safety monitoring by identifying invalid signals,communication interruptions, or unexpected command conditions
- For example:
  - Waste management issues → Sanitation department
  - Road damage → Civil/Engineering department

3.2.2 Command Validation and Safety Method

- Before transmitting a command, the system validates the processed EMG signal to ensure that it represents a meaningful muscle activation.
- Calibration thresholds are used to distinguish intentional muscle activity from resting conditions and unwanted noise.

3.2.3 MARTS Overall Algorithm

Input: Raw EMG signals collected from the user's muscles through EMG sensors.
Output: Classified control commands transmitted to the drone for navigation and movement.

Step 1: Start
Step 2: Initialize the system

- Initialize the EMG sensor and microcontroller.
- Establish wireless communication between the user-side controller and drone receiver.

Step 3: Acquire EMG signals
3.1 Read electrical signals generated by muscle contractions.
3.2 Collect raw EMG data from the sensors.
3.3 Send the acquired signals to the signal conditioning unit.
Step 4: Signal conditioning and preprocessing
4.1 Amplify the acquired EMG signals.
4.2 Apply filtering to reduce unwanted noise and interference.
4.3 Prepare the filtered signals for further processing.
Step 5: EMG signal processing
5.1 Analize muscle activation level
Step 6: Command classification
6.1 Identify left-side muscle activation → Generate LEFT command.
6.2 Identify right-side muscle activation → Generate RIGHT command.
6.3 Detect resting muscle condition → Generate STOP command.
Step7: CommandValidation
7.1 Validate the

classified command

Step 8: Wireless communication

Step 9: Drone control and movement

Step 10: Feedback and monitoring

Step 11: Repeat the process

Step 12: End

3.3 Project Architecture

3.3.1 Architectural Diagram

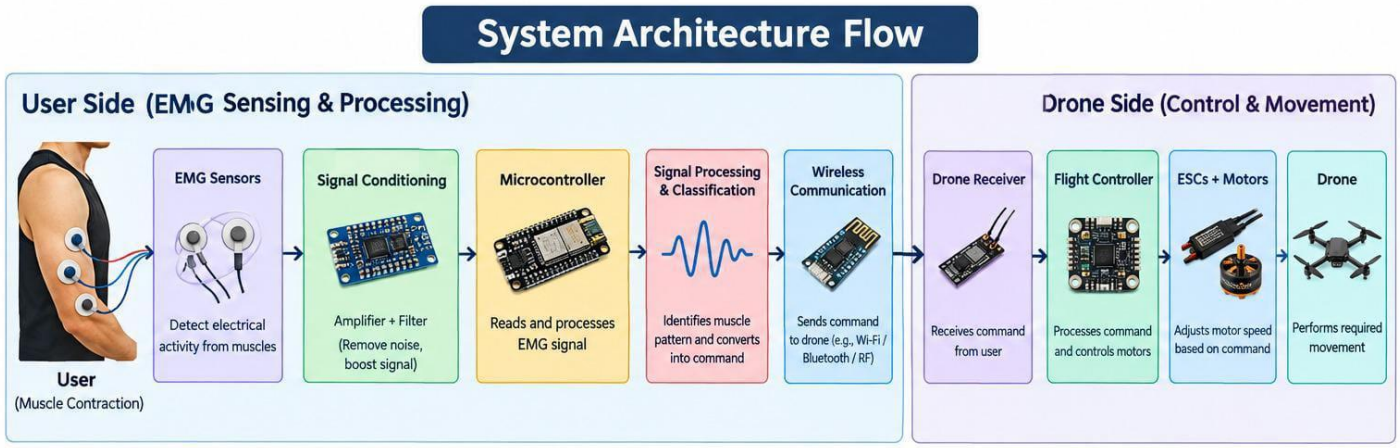


Figure 1: Architeture Diagram

The **user side** of the **NeuroFlight** project collects muscle signals using EMG sensors attached to the user's arm. These sensors detect electrical signals produced when the muscles contract. The signal conditioning unit amplifies the signals and removes unwanted noise. The microcontroller reads and processes the signals, and the signal processing unit identifies muscle patterns and converts them into commands for controlling the drone.

After processing, the commands are sent wirelessly to the drone using Wi-Fi, Bluetooth, or RF communication. The drone receiver receives these commands and sends them to the flight controller. The flight controller processes the commands and controls the motors through Electronic Speed Controllers (ESCs). The ESCs adjust the motor speed according to the received commands, helping the drone move in the required direction.

The NeuroFlight system connects human muscle movements with drone navigation. When the user contracts specific muscles, the EMG sensors detect the signals and send them for processing. The processed signals are converted into movement commands and transmitted wirelessly to the drone. The drone receives these commands and performs the required movements. This system provides a simple and innovative way to control a drone using muscle signals instead of a traditional remote controller.

3.3.2 Comprehensive Flow Diagram

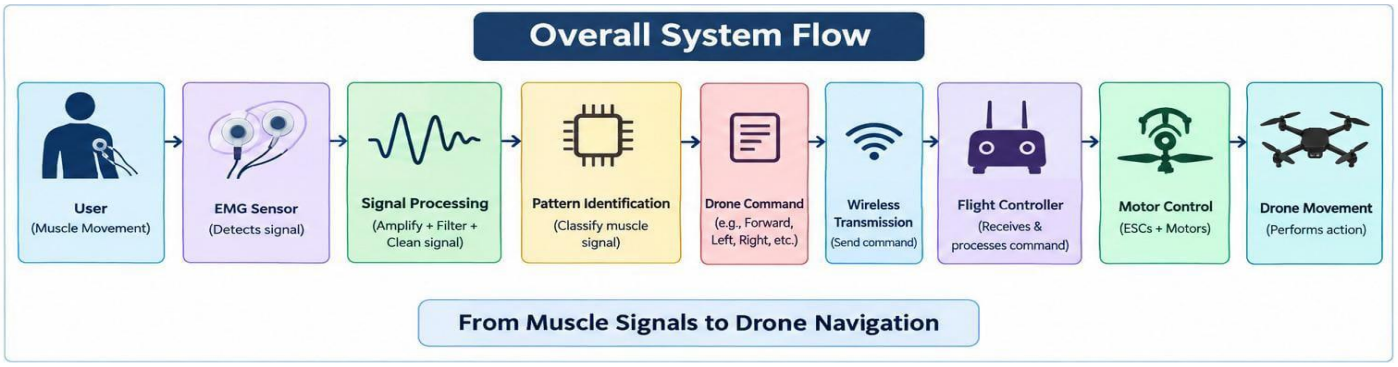


Figure 2: Comprehensive Data Flow Diagram

The overall system flow of the NeuroFlight project explains how a drone can be controlled using human muscle movements. First, EMG sensors detect the electrical signals produced when the user contracts their muscles. These signals are amplified, filtered, and processed to remove unwanted noise. The system then identifies the muscle patterns and converts them into commands, such as moving forward, backward, left, or right. These commands are transmitted wirelessly to the drone, where the flight controller receives and processes them. The motor control system adjusts the motor speeds according to the commands, allowing the drone to perform the required movements. Overall, this project provides an innovative way to control a drone using muscle signals instead of a traditional remote controller, making drone navigation more interactive and hands-free. The NeuroFlight system uses EMG sensors, signal processing, and wireless communication to connect human muscle movements with drone control. It converts muscle signals into commands and helps the drone move in the desired direction. This project offers a simple and innovative approach to hands-free drone navigation.

3.3.3 Class Diagram

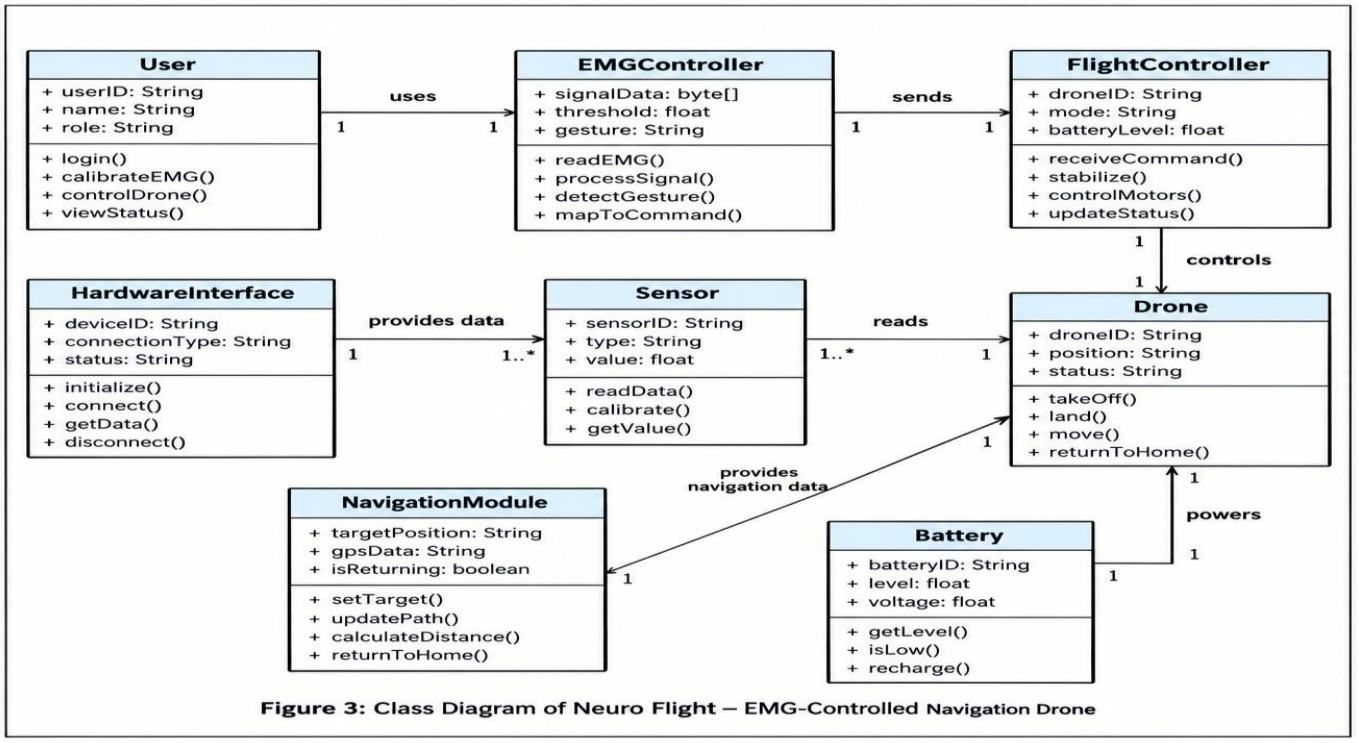


Figure 3: Class Diagram of Neuro Flight – EMG-Controlled Navigation Drone

The class diagram represents the main classes and their relationships in the NeuroFlight project. The User interacts with the EMGController, which detects muscle signals and converts them into commands. The FlightController receives these commands and controls the drone's movements. The Sensors collect data required for navigation, while the Navigation Module helps determine the target position and path. The Battery supplies power to the drone and monitors its power level. The HardwareInterface manages communication between the EMG device and other hardware components. Overall, these classes work together to control the drone using human muscle signals and achieve efficient drone navigation.

3.3.4 Use Case Diagram

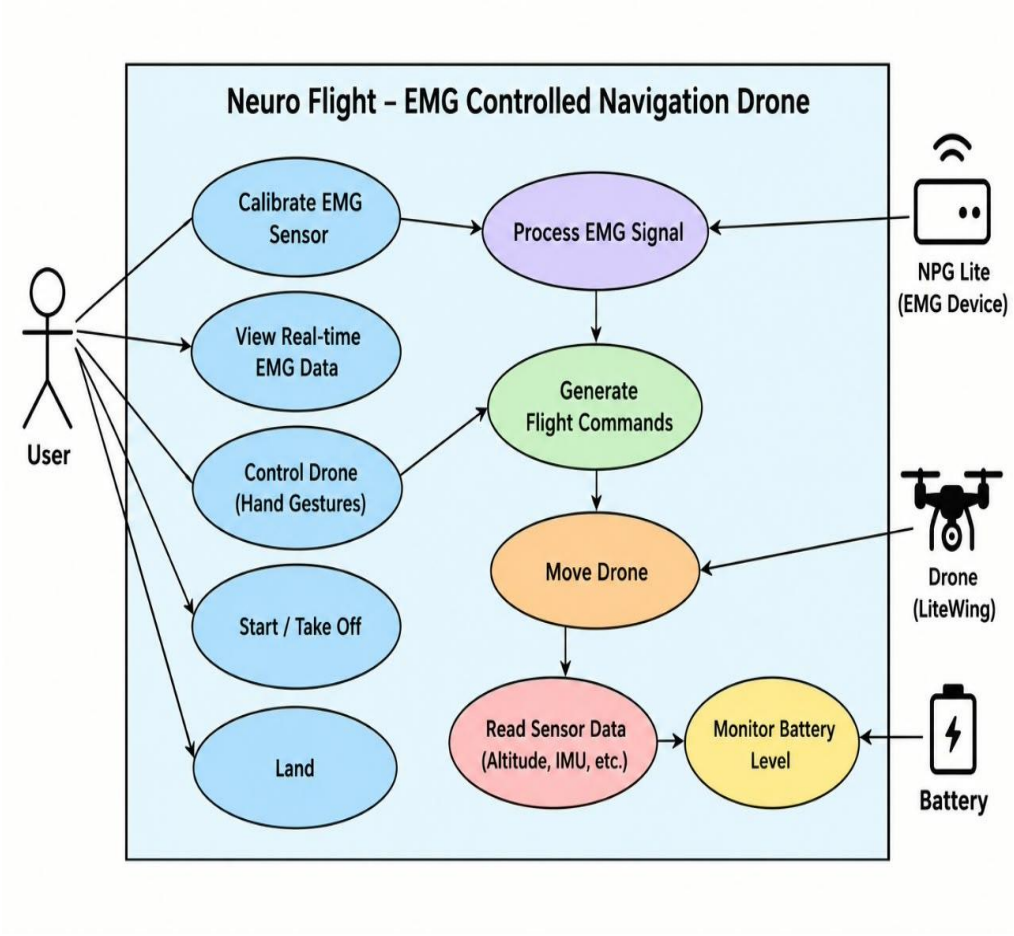


Figure 4: Use Case Diagram of Neuro Flight-EMG Controlled Navigation Drone

The use case diagram explains how the user interacts with the NeuroFlight project to control a drone using EMG signals. The user can calibrate the EMG sensor, view real-time muscle data, control the drone using hand gestures, and perform take-off and landing operations. The NPG Lite EMG device collects muscle signals and sends them to the system for processing. The system converts the processed signals into flight commands and transmits them to the drone to perform the required movements. It also reads sensor data, such as altitude and IMU values, and monitors the battery level to support safe and efficient drone navigation.

3.3.5 Sequence Diagram

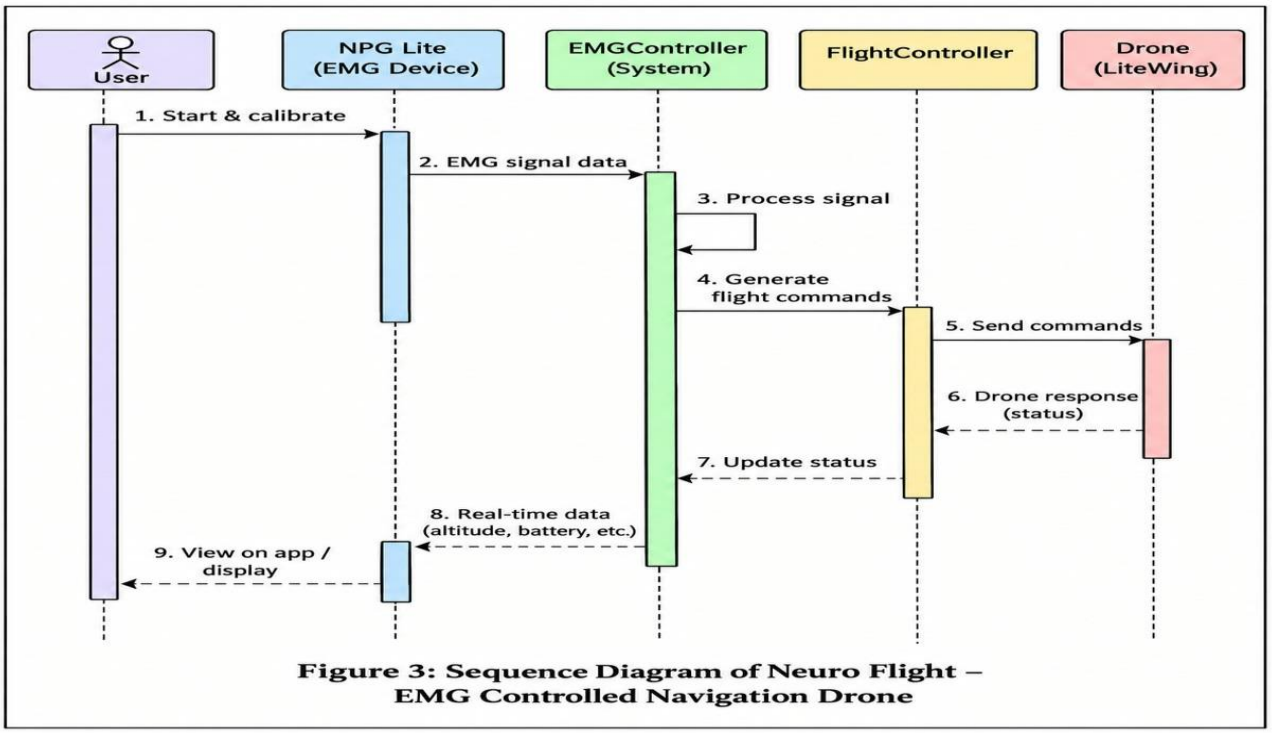


Figure 3: Sequence Diagram of Neuro Flight – EMG Controlled Navigation Drone

Figure 5: Sequence Diagram Of Neuro Flight – EMG Controlled Navigation Drone

The sequence diagram explains the step-by-step interaction between the user and the different components of the Neuro Flight project. First, the user starts the system and calibrates the NPG Lite EMG device. The device collects muscle signals and sends them to the EMG Controller, which processes the signals and generates flight commands. These commands are sent to the Flight Controller, which controls the LiteWing drone. The drone performs the required movement and sends its status back to the system. Finally, the system updates and displays real-time information, such as altitude and battery level, on the app or display. This process helps the user control the drone using muscle signals and monitor its performance.

3.3.6 Activity Diagram

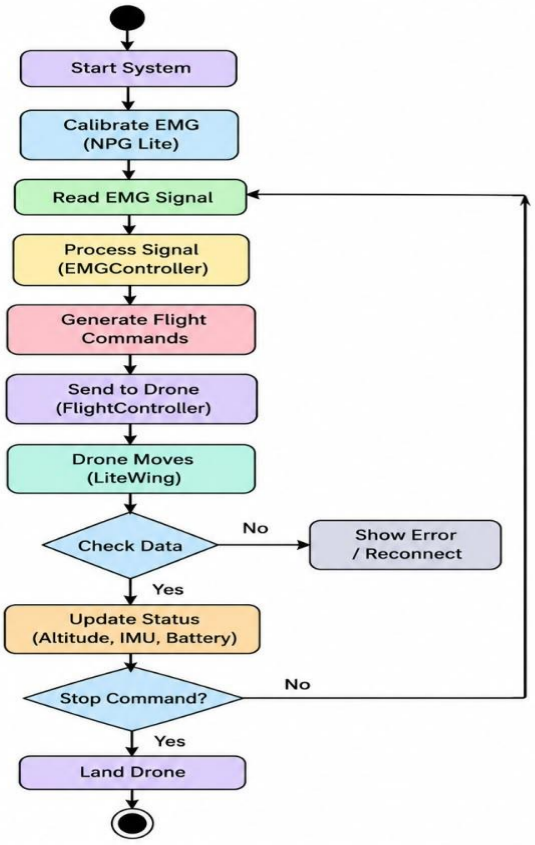


Figure 3: Activity Diagram of Neuro Flight – EMG Controlled Navigation Drone

Figure 6: Comprehensive Activity Diagram for Municipal Action & Response Tracking System

The activity diagram explains the step-by-step working of the NeuroFlight project. First, the system starts and calibrates the EMG sensor to detect muscle signals. The signals are read and processed by the EMGController, which converts them into flight commands. These commands are sent to the FlightController, which controls the drone's movement. After the drone moves, the system checks the data and updates information such as altitude, IMU readings, and battery level. If an error occurs, the system shows an error message or reconnects. This process continues until the user gives a stop command, after which the drone lands safely and the system ends.

CHAPTER 4

IMPLEMENTATION & TESTING

4.1 Coding Blocks

4.1.1 Main Functional block:

Live dashboard: NPG Lite EMG ->
LiteWing motor-output bench test.

SAFETY:
- REMOVE ALL PROPELLERS before running.
- Direct motorPowerSet output is NOT normal flight control.
- Motor power remains at the previously tested value 1000.
- Each activation is limited to a short pulse.
- This does not provide directional flight or controlled yaw.

Run from module-npg:
    uv run python integrated_emg_dashboard.py
"""

```
import asyncio
import math
import queue
import statistics
import threading
import time
import tkinter as tk
from tkinter import ttk
from collections import deque

import numpy as np
from scipy.signal import butter, iirnotch, sosfilt, lfilter

from bleak import BleakClient
import cflib.crtp
from cflib.crazyflie import Crazyflie


# ---------------- DEVICE SETTINGS ----------------
NPG_MAC = "E4:B3:23:AF:72:B6"
NPG_DATA_UUID = "beb5483e-36e1-4688-b7f5-ea07361b26a8"
NPG_CONTROL_UUID = "0000ff01-0000-1000-8000-00805f9b34fb"
LITEWING_URI = "udp://192.168.43.42:2390"

TEST_POWER = 1000
MAX_MOTOR_PULSE_SECONDS = 1.0
CALIBRATION_SECONDS = 3.0
SAMPLE_RATE_HZ = 500
```

```
SAMPLE_BYTES = 7
RMS_WINDOW_SAMPLES = 25
THRESHOLD_MULTIPLIER = 2.5
THRESHOLD_FLOOR = 20.0
RELEASE_RATIO = 0.65

# ---------- SHARED STATE ----------
cf = None
cf_connected = threading.Event()
shutdown_requested = threading.Event()
motor_queue = queue.Queue()
ui_queue = queue.Queue()
motor_worker_thread = None

packet_buffer = bytearray()
calibration_samples = [[], []]
rms_windows = [deque(maxlen=RMS_WINDOW_SAMPLES),
        deque(maxlen=RMS_WINDOW_SAMPLES)]
baseline = [None, None]
noise_rms = [None, None]
activation_threshold = [None, None]
calibration_start = None
calibration_done = False
pulse_latched_until_release = False
motor_pulse_started = None
last_motor_request = False
last_status_time = 0.0
motor_active_until = 0.0
latest_status = {
   "ble": "Waiting",
   "wifi": "Waiting",
   "phase": "Waiting to start",
   "left_raw": 0,
   "right_raw": 0,
   "left_rms": 0.0,
   "right_rms": 0.0,
   "command": "STOP",
   "motor": "OFF",
   "filter": "Bandpass 20–100 Hz + 50 Hz notch",
   "message": "Remove propellers before starting.",
}

# Streaming causal filters at 500 Hz: bandpass then notch on each channel.
BP_SOS = butter(4, [20, 100], btype="bandpass", fs=SAMPLE_RATE_HZ, output="sos")
NOTCH_B, NOTCH_A = iirnotch(50, 30, fs=SAMPLE_RATE_HZ)
filter_sos_state = [np.zeros((BP_SOS.shape[0], 2)), np.zeros((BP_SOS.shape[0], 2))]
notch_state = [np.zeros(max(len(NOTCH_A), len(NOTCH_B)) - 1),
        np.zeros(max(len(NOTCH_A), len(NOTCH_B)) - 1)]


def ui_update(**kwargs):
   latest_status.update(kwargs)
```

```
    ui_queue.put(dict(latest_status))


def on_connected(uri):
    cf_connected.set()
    ui_update(wifi="Connected", message=f"LiteWing connected: {uri}")


def on_connection_failed(uri, message):
    ui_update(wifi="Failed", message=f"LiteWing connection failed: {message}")


def on_disconnected(uri):
    cf_connected.clear()
    shutdown_requested.set()
    ui_update(wifi="Disconnected", command="STOP", motor="OFF",
              message="LiteWing disconnected; stopping.")


def set_param(name, value):
    if cf is None or not cf_connected.is_set():
        raise ConnectionError("LiteWing is not connected.")
    cf.param.set_value(name, str(value))
    time.sleep(0.15)


def stop_motor_outputs():
    if cf is None or not cf_connected.is_set():
        return
    for motor in ("m1", "m2", "m3", "m4"):
        try:
            set_param(f"motorPowerSet.{motor}", 0)
        except Exception as exc:
            ui_update(message=f"Could not zero {motor}: {exc}")
    try:
        set_param("motorPowerSet.enable", 0)
    except Exception as exc:
        ui_update(message=f"Could not disable motor output: {exc}")


def start_motor_outputs():
    if cf is None or not cf_connected.is_set():
        raise ConnectionError("LiteWing is disconnected.")
    set_param("motorPowerSet.enable", 0)
    for motor in ("m1", "m2", "m3", "m4"):
        set_param(f"motorPowerSet.{motor}", TEST_POWER)
    set_param("motorPowerSet.enable", 1)


def motor_worker():
    output_on = False
    while True:
```

```
        command = motor_queue.get()
        try:
            if command is None:
                break
            if command == "ON" and not output_on:
                try:
                    start_motor_outputs()
                    output_on = True
                    ui_update(motor="ON (pulse)", message=f"Muscle flex detected; outputs at
{TEST_POWER}.")
                except Exception as exc:
                    stop_motor_outputs()
                    output_on = False
                    ui_update(motor="OFF", command="STOP", message=f"Motor activation failed: {exc}")
            elif command == "OFF":
                stop_motor_outputs()
                output_on = False
                ui_update(motor="OFF")
        finally:
            motor_queue.task_done()
    stop_motor_outputs()


def request_motor(on):
    global last_motor_request
    if on == last_motor_request:
        return
    last_motor_request = on
    motor_queue.put("ON" if on else "OFF")


def parse_sample(raw):
    return (raw[0],
            int.from_bytes(raw[1:3], "big"),
            int.from_bytes(raw[3:5], "big"),
            int.from_bytes(raw[5:7], "big"))


def filter_emg(channel, raw_value):
    # Remove DC baseline approximately, then stream through bandpass and notch.
    # Initial state is zero; brief startup transients are expected.
    x = float(raw_value)
    y_bp, filter_sos_state[channel][:] = sosfilt(
        BP_SOS, np.array([x]), zi=filter_sos_state[channel]
    )
    y_notch, notch_state[channel][:] = lfilter(
        NOTCH_B, NOTCH_A, y_bp, zi=notch_state[channel]
    )
    return float(y_notch[0])


def finish_calibration():
```

```
    global calibration_done
    if len(calibration_samples[0]) < SAMPLE_RATE_HZ:
        ui_update(phase="Calibration failed", message="Not enough samples; keep outputs stopped.")
        return
    for channel in (0, 1):
        vals = calibration_samples[channel]
        center = statistics.median(vals)
        baseline[channel] = center
        noise = math.sqrt(sum((v - center) ** 2 for v in vals) / len(vals))
        noise_rms[channel] = noise
        activation_threshold[channel] = max(
            noise * THRESHOLD_MULTIPLIER, noise + THRESHOLD_FLOOR
        )
    calibration_done = True
    ui_update(phase="Live monitoring",
              message="Calibration complete. Flex one forearm briefly, then relax.")


def process_sample(counter, ch0, ch1):
    global calibration_start, pulse_latched_until_release
    global motor_pulse_started, last_status_time

    now = time.monotonic()
    raw_values = (ch0, ch1)
    filtered = (filter_emg(0, ch0), filter_emg(1, ch1))

    if calibration_start is None:
        calibration_start = now
        ui_update(phase="Calibrating (keep relaxed)", message="Keep both forearms relaxed for 3
seconds.")

    if not calibration_done:
        for channel in (0, 1):
            calibration_samples[channel].append(filtered[channel])
        if now - calibration_start >= CALIBRATION_SECONDS:
            finish_calibration()
        return

    # Use filtered signal deviation from the relaxed filtered baseline.
    for channel in (0, 1):
        rms_windows[channel].append((filtered[channel] - baseline[channel]) ** 2)
    if any(len(w) < RMS_WINDOW_SAMPLES for w in rms_windows):
        return

    rms = [math.sqrt(sum(w) / len(w)) for w in rms_windows]
    left_active = rms[0] >= activation_threshold[0]
    right_active = rms[1] >= activation_threshold[1]
    fully_released = (
        rms[0] < activation_threshold[0] * RELEASE_RATIO
        and rms[1] < activation_threshold[1] * RELEASE_RATIO
    )
```

```
    if left_active and right_active:
        command = "BOTH ACTIVE — STOP"
    elif left_active:
        command = "LEFT EMG ACTIVATED"
    elif right_active:
        command = "RIGHT EMG ACTIVATED"
    else:
        command = "NO MUSCLE ACTIVATION"

    ui_update(
        left_raw=raw_values[0],
        right_raw=raw_values[1],
        left_rms=round(rms[0], 1),
        right_rms=round(rms[1], 1),
        command=command,
        phase="Live monitoring",
    )

    if fully_released:
        pulse_latched_until_release = False

    # Bench test only: either isolated muscle flex triggers a short pulse.
    # Both channels active is deliberately treated as STOP.
    if left_active and right_active:
        request_motor(False)
        motor_pulse_started = None
    elif (left_active or right_active) and not pulse_latched_until_release:
        pulse_latched_until_release = True
        motor_pulse_started = now
        request_motor(True)

    if motor_pulse_started is not None and now - motor_pulse_started >=
MAX_MOTOR_PULSE_SECONDS:
        request_motor(False)
        motor_pulse_started = None
        ui_update(message="Pulse time limit reached; relax before triggering again.")


def on_npg_notification(sender, data):
    if shutdown_requested.is_set():
        return
    packet_buffer.extend(data)
    while len(packet_buffer) >= SAMPLE_BYTES:
        raw = bytes(packet_buffer[:SAMPLE_BYTES])
        del packet_buffer[:SAMPLE_BYTES]
        try:
            counter, ch0, ch1, ch2 = parse_sample(raw)
            process_sample(counter, ch0, ch1)
        except Exception as exc:
            request_motor(False)
            ui_update(command="STOP", motor="OFF", message=f"EMG processing error: {exc}")
```

```
async def write_npg_command(client, command):
    payload = command.encode("ascii")
    try:
        await client.write_gatt_char(NPG_CONTROL_UUID, payload, response=True)
    except Exception:
        await client.write_gatt_char(NPG_CONTROL_UUID, payload, response=False)


async def hardware_main():
    global cf, motor_worker_thread
    try:
        ui_update(phase="Connecting LiteWing", wifi="Connecting",
                  message="Remove all propellers. Connecting to LiteWing over Wi-Fi.")
        cflib.crtp.init_drivers()
        cf = Crazyflie(rw_cache="./cache")
        cf.connected.add_callback(on_connected)
        cf.connection_failed.add_callback(on_connection_failed)
        cf.disconnected.add_callback(on_disconnected)
        motor_worker_thread = threading.Thread(target=motor_worker, daemon=True)
        motor_worker_thread.start()
        cf.open_link(LITEWING_URI)

        if not cf_connected.wait(timeout=15):
            raise TimeoutError("LiteWing connection timed out.")
        await asyncio.sleep(8)
        await asyncio.to_thread(stop_motor_outputs)

        ui_update(ble="Connecting", phase="Connecting NPG Lite",
                  message="Connecting to NPG Lite over BLE.")
        async with BleakClient(NPG_MAC, timeout=20.0) as client:
            if not client.is_connected:
                raise ConnectionError("NPG Lite BLE connection failed.")
            ui_update(ble="Connected", phase="Starting EMG stream",
                      message="NPG Lite connected; starting notifications.")
            await client.start_notify(NPG_DATA_UUID, on_npg_notification)
            await write_npg_command(client, "START")
            ui_update(phase="Calibrating (keep relaxed)",
                      message="Keep both forearms relaxed for 3 seconds.")
            while not shutdown_requested.is_set() and client.is_connected:
                await asyncio.sleep(0.1)
            try:
                await write_npg_command(client, "STOP")
            except Exception:
                pass
            try:
                await client.stop_notify(NPG_DATA_UUID)
            except Exception:
                pass
    except Exception as exc:
        ui_update(phase="Error", command="STOP", motor="OFF",
                  message=f"Connection/runtime error: {exc}")
```

```
    finally:
        shutdown_requested.set()
        try:
            motor_queue.put("OFF")
            motor_queue.join()
            if motor_worker_thread:
                motor_queue.put(None)
                motor_queue.join()
                motor_worker_thread.join(timeout=2)
        except Exception:
            pass
        try:
            await asyncio.to_thread(stop_motor_outputs)
        except Exception:
            pass
        if cf is not None:
            try:
                cf.close_link()
            except Exception:
                pass
        ui_update(phase="Stopped", command="STOP", motor="OFF",
                  message="Program stopped. Verify motor outputs are off.")


def run_hardware_loop():
    asyncio.run(hardware_main())


# ---------------- SIMPLE LIVE DASHBOARD ----------------
def launch_dashboard():
    root = tk.Tk()
    root.title("NPG Lite + LiteWing | EMG Dashboard")
    root.geometry("760x560")
    root.minsize(700, 520)

    style = ttk.Style(root)
    try:
        style.theme_use("clam")
    except tk.TclError:
        pass

    outer = ttk.Frame(root, padding=18)
    outer.pack(fill="both", expand=True)

    ttk.Label(outer, text="EMG DRONE CONTROL", font=("Segoe UI", 18, "bold")).pack(anchor="w")
    ttk.Label(outer, text="Live monitor • Propellers must remain removed",
              font=("Segoe UI", 10)).pack(anchor="w", pady=(2, 14))

    connection_frame = ttk.LabelFrame(outer, text="Connections", padding=10)
    connection_frame.pack(fill="x", pady=(0, 10))
    ble_var = tk.StringVar(value="Waiting")
    wifi_var = tk.StringVar(value="Waiting")
```

```
    ttk.Label(connection_frame, text="NPG Lite BLE:", width=18).grid(row=0, column=0, sticky="w",
padx=4, pady=3)
    ttk.Label(connection_frame, textvariable=ble_var, font=("Segoe UI", 10, "bold")).grid(row=0,
column=1, sticky="w", padx=4, pady=3)
    ttk.Label(connection_frame, text="LiteWing Wi-Fi:", width=18).grid(row=1, column=0, sticky="w",
padx=4, pady=3)
    ttk.Label(connection_frame, textvariable=wifi_var, font=("Segoe UI", 10, "bold")).grid(row=1,
column=1, sticky="w", padx=4, pady=3)

    readings = ttk.Frame(outer)
    readings.pack(fill="x", pady=(0, 10))
    left_box = ttk.LabelFrame(readings, text="LEFT EMG (Channel 0)", padding=12)
    right_box = ttk.LabelFrame(readings, text="RIGHT EMG (Channel 1)", padding=12)
    left_box.pack(side="left", fill="both", expand=True, padx=(0, 6))
    right_box.pack(side="left", fill="both", expand=True, padx=(6, 0))
    left_raw_var, right_raw_var = tk.StringVar(value="Raw: —"), tk.StringVar(value="Raw: —")
    left_rms_var, right_rms_var = tk.StringVar(value="Filtered RMS: —"), tk.StringVar(value="Filtered
RMS: —")
    ttk.Label(left_box, textvariable=left_raw_var, font=("Segoe UI", 12)).pack(anchor="w", pady=4)
    ttk.Label(left_box, textvariable=left_rms_var, font=("Segoe UI", 11)).pack(anchor="w", pady=4)
    ttk.Label(right_box, textvariable=right_raw_var, font=("Segoe UI", 12)).pack(anchor="w", pady=4)
    ttk.Label(right_box, textvariable=right_rms_var, font=("Segoe UI", 11)).pack(anchor="w", pady=4)

    command_frame = ttk.LabelFrame(outer, text="Command classification", padding=12)
    command_frame.pack(fill="x", pady=(0, 10))
    command_var = tk.StringVar(value="STOP")
    command_label = ttk.Label(command_frame, textvariable=command_var,
                              font=("Segoe UI", 17, "bold"), anchor="center")
    command_label.pack(fill="x", pady=5)

    motor_frame = ttk.LabelFrame(outer, text="Motor output", padding=10)
    motor_frame.pack(fill="x", pady=(0, 10))
    motor_var = tk.StringVar(value="OFF")
    ttk.Label(motor_frame, textvariable=motor_var, font=("Segoe UI", 12, "bold")).pack(anchor="w")

    phase_var = tk.StringVar(value="Waiting to start")
    message_var = tk.StringVar(value="Remove all propellers before running.")
    ttk.Label(outer, textvariable=phase_var, font=("Segoe UI", 10, "bold")).pack(anchor="w", pady=(2, 2))
    ttk.Label(outer, textvariable=message_var, wraplength=700).pack(anchor="w")

    ttk.Label(outer, text="Filtering: 20–100 Hz bandpass + 50 Hz notch; RMS activation detector.",
              font=("Segoe UI", 9)).pack(anchor="w", pady=(10, 0))

    def poll_updates():
        while True:
            try:
                state = ui_queue.get_nowait()
            except queue.Empty:
                break
            ble_var.set(state.get("ble", "Waiting"))
            wifi_var.set(state.get("wifi", "Waiting"))
            left_raw_var.set(f"Raw: {state.get('left_raw', 0)}")
```

```
            right_raw_var.set(f"Raw: {state.get('right_raw', 0)}")
            left_rms_var.set(f"Filtered RMS: {state.get('left_rms', 0)}")
            right_rms_var.set(f"Filtered RMS: {state.get('right_rms', 0)}")
            command_var.set(state.get("command", "STOP"))
            motor_var.set(state.get("motor", "OFF"))
            phase_var.set(state.get("phase", "Waiting"))
            message_var.set(state.get("message", ""))
        if not shutdown_requested.is_set():
            root.after(100, poll_updates)

    def close_window():
        shutdown_requested.set()
        try:
            motor_queue.put("OFF")
        except Exception:
            pass
        root.destroy()

    root.protocol("WM_DELETE_WINDOW", close_window)
    threading.Thread(target=run_hardware_loop, name="hardware-loop", daemon=True).start()
    root.after(100, poll_updates)
    root.mainloop()


if __name__ == "__main__":
    print("SAFETY CHECK: remove all propellers before running.")
    print("This dashboard uses direct motor output for a short bench pulse only.")
    input("Press ENTER after confirming all propellers are removed...")
    try:
        launch_dashboard()
    except KeyboardInterrupt:
        shutdown_requested.set()
```

Github link: https://github.com/Gsaiteja25/marts

4.2 Test Case Study

4.2.1 Test Case 1

EMG Signal Acquisition

Test Scenario: Verify EMG signal acquisition from the NPG Lite device.

Objective: Ensure that the system successfully receives raw EMG signals from the NPG Lite device through the Bluetooth connection.

Preconditions:

- NPG Lite device is powered ON.
- EMG electrodes are properly attached to the user's muscles.
- Bluetooth is enabled and the device is available.

Test Steps:

- Power ON the NPG Lite device.

- Connect the device to the NeuroFlight application.
- Place the electrodes on the selected muscle locations.
- Observe the raw EMG signal values on the dashboard.

Expected Result:

- The NPG Lite device should connect successfully.
- Raw EMG signals should be received continuously.
- Signal values should change according to muscle activity.

4.2.2 Test Case 2

Objective: Ensure that unwanted noise is removed from the raw EMG signal to obtain a clean signal for accurate muscle activity detection.

Preconditions:

- NPG Lite device is connected.
- Raw EMG signals are being received.
- Signal processing module is running.

Test Steps:

- Acquire raw EMG signals from the NPG Lite device.
- Apply a 20–100 Hz band-pass filter.
- Apply a 50 Hz notch filter to reduce power-line interference.
- Calculate the filtered RMS values.
- Display the processed signal values on the dashboard.

Expected Result:

- Unwanted noise should be reduced.
- Filtered EMG signals should be smoother.
- RMS values should represent muscle activity accurately.
- Actual Result: To be recorded after verifying the filtered EMG output.

Status: Pending

Postconditions: Filtered EMG signals are available for muscle contraction detection.

4.2.3 Test Case 3

Objective: Ensure that the system detects muscle contractions based on the amplitude and RMS values of the processed EMG signals.

Preconditions:

- EMG signal acquisition is working.
- Signal filtering is completed.
- Contraction detection thresholds are configured.

Test Steps:

- Keep the muscles in a relaxed state.
- Observe the baseline EMG and RMS values.
- Contract the left-side muscle.
- Observe the corresponding change in signal amplitude and RMS.
- Repeat the process with the right-side muscle.

Expected Result:

- The system should identify the resting muscle condition.
- Muscle contraction should produce an increase in RMS values.
- Left and right muscle activities should be detected separately.
- Actual Result: To be recorded after testing with actual EMG signals.

Status: Pending

Postconditions: Muscle activity information is forwarded to the classification module.

4.2.4 Test Case 4

Objective: Ensure that the system detects muscle contractions based on the amplitude and RMS values of the processed EMG signals.

Preconditions:

- EMG signal acquisition is working.
- Signal filtering is completed.
- Contraction detection thresholds are configured.

Test Steps:

- Keep the muscles in a relaxed state.
- Observe the baseline EMG and RMS values.
- Contract the left-side muscle.
- Observe the corresponding change in signal amplitude and RMS.
- Repeat the process with the right-side muscle.

Expected Result:

- The system should identify the resting muscle condition.
- Muscle contraction should produce an increase in RMS values.
- Left and right muscle activities should be detected separately.

Actual Result: To be recorded after testing with actual EMG signals.

Status: Pending

Postcondition **s:** Muscle activity information is forwarded to the classification module.

CHAPTER 5 RESULTS

5.1 Resulting Screens

5.1.1 Dashboard



Figure 7: User Dashboard

The dashboard displays the left and right EMG sensor readings, filtered RMS values, connection status, classified control commands, and motor output status

5.2 Resulting Tables

5.3 Table 1 Existing system vs neuro flight system

Component	Avg Response Time	CPU Usage	Memory Usage	Status
NPG Lite BLE	To be measured	To be measured	To be measured	Waiting
EMG Signal Processing	To be measured	To be measured	To be measured	Implemented
Muscle Contraction Detection	To be measured	To be measured	To be measured	Implemented
EMG Command Classification	To be measured	To be measured	To be measured	Implemented
LiteWing Wi-Fi Communication	To be measured	To be measured	To be measured	Connecting

Table 1 Existing system vs neuro flight system

This table represents the performance of different NeuroFlight components,

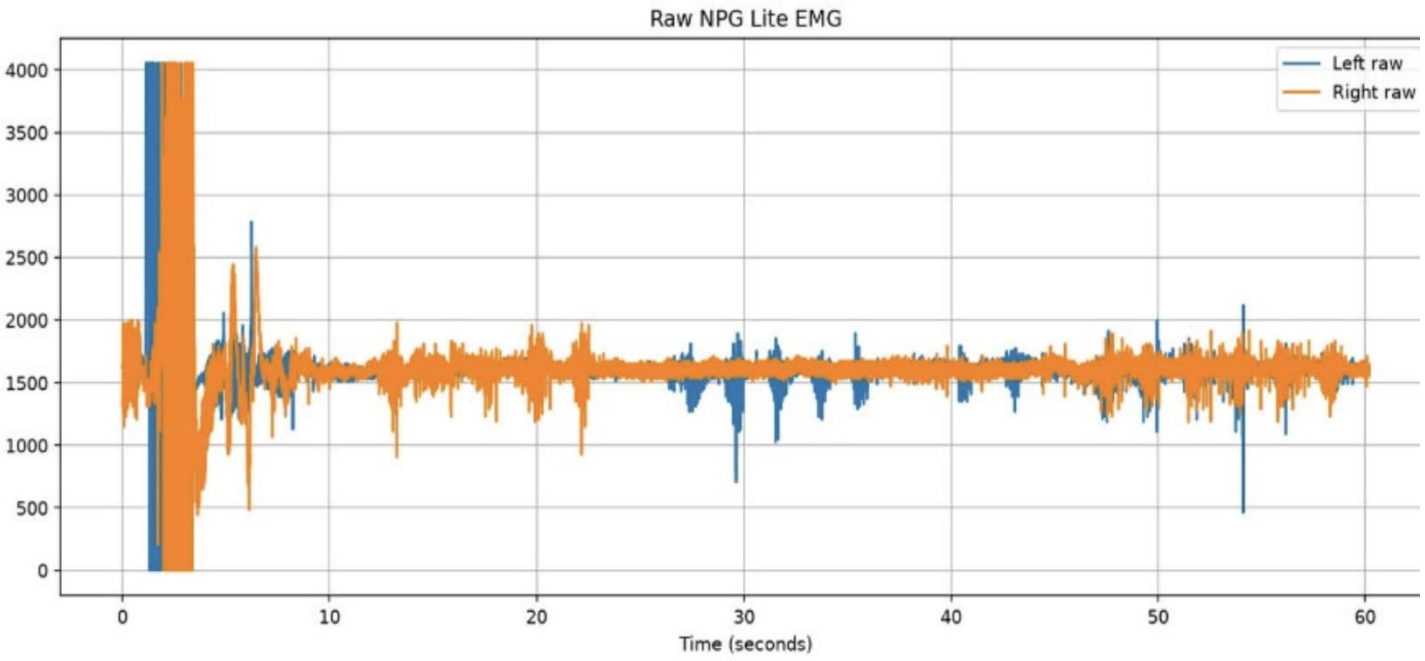
Feature	**Existing System**	**NeuroFlight System**
Control Method	Manual Remote Control	EMG-Based Control
Signal Acquisition	Not Required	NPG Lite EMG
Signal Processing	Not Required	Digital Filtering and RMS
Muscle Detection	Not Available	EMG Contraction Detection
Command Generation	Manual Input	EMG Command Classification
Drone Communication	Remote Controller	ESP32-S3 Wi-Fi
Navigation Commands	Joystick-Based	LEFT, RIGHT, STOP

Table 2 Existing System vs NeuroFlight System Comparison

This table compares conventional drone control systems with the proposed NeuroFlight system. Unlike traditional remote-controlled drones, NeuroFlight uses EMG signals to detect muscle contractions and generate navigation commands. It also provides real-time monitoring of EMG signals, command classification, device connections, and motor output through a dashboard.

5.4 Resulting graphs

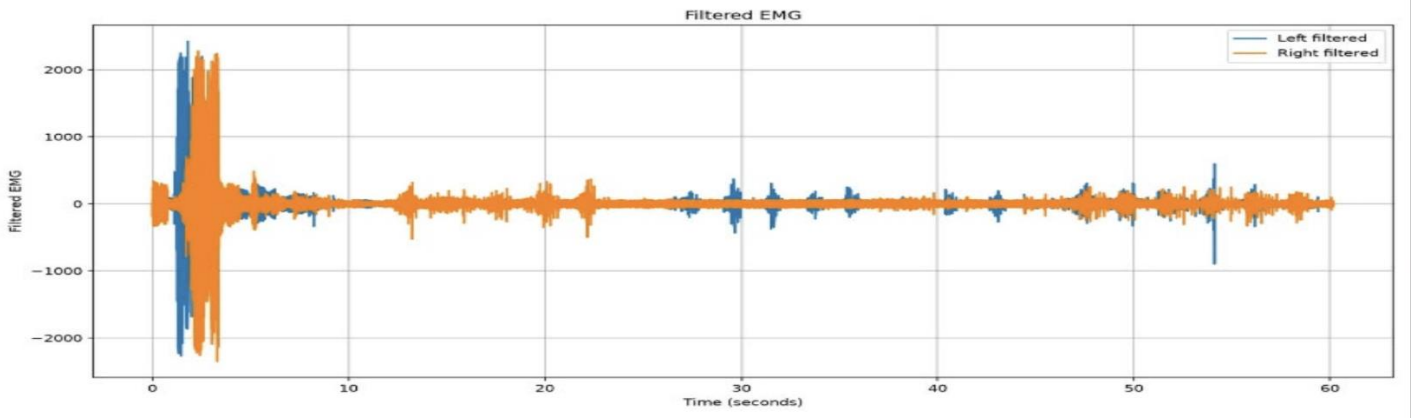
5.4.1 Result graph 1: Raw NPG Lite EMG



Graph 1: Raw NPG Lite EMG

Raw NPG Lite EMG Signal: It represents the original electrical signal detected by the EMG electrodes before digital filtering and feature extraction.

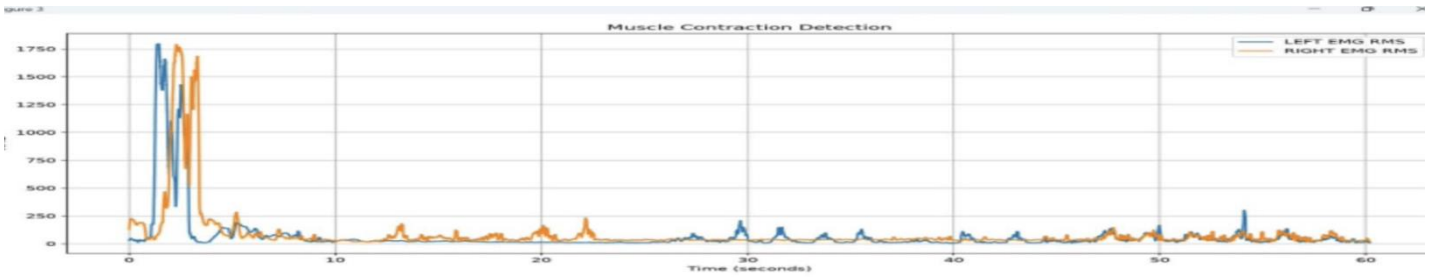
5.4.2 Result graph 2: Filtered EMG



Graph 2: Filtered EMG

Filtered EMG is the EMG signal after unwanted noise and interference have been reduced using digital filtering techniques.

5.4.3 Result graph 3:Muscle Contraction Detection



Graph 3: Muscle Contraction Detection graph

Muscle contraction detection is the process of identifying when a user's muscle becomes active by analyzing the electrical signals generated during muscle contraction.

5.5 Results Analysis

5.5.1 Time Complexity

The time complexity of the NeuroFlight system depends on how efficiently the EMG signals are collected, processed, and converted into drone control commands. The ESP32 microcontroller processes the incoming sensor values and compares them with predefined signal thresholds to identify muscle movements. These operations generally require constant time, O(1), for each sensor reading when a fixed number of conditions is checked.

The system is designed to process muscle signals continuously and send commands to the drone with minimal delay. Efficient signal processing and wireless communication help improve the response time of the system. The overall performance depends on the processing algorithm, sensor sampling rate, and communication speed. Therefore, an optimized implementation helps achieve faster responses and smoother drone navigation.

5.5.2 Space Complexity

The space complexity of the NeuroFlight system depends on the memory required to store EMG sensor readings, processed signal values, and drone control commands. The ESP32 microcontroller uses its memory to store program instructions, variables, and temporary data during operation. Since the system processes sensor readings continuously without storing large amounts of data, the memory requirement can be kept low.

The system can operate efficiently by using a fixed amount of memory for signal processing and command generation. If only a few sensor values and control variables are stored at a time, the space complexity is O(1). This helps reduce memory usage and allows the ESP32 to perform real-time operations efficiently.

5.5.3 Results Summary

The NeuroFlight: EMG-Controlled Drone Navigation project demonstrates an alternative method of controlling a drone using human muscle signals. The EMG sensor detects electrical activity from the user's muscles, and the ESP32 microcontroller processes these signals into control commands. These commands are transmitted to the drone through a suitable communication system to perform the required movements. The system combines biomedical sensors, embedded systems, and wireless communication.

The project provides an understanding of how muscle signals can be used for human–machine interaction and drone navigation. Its performance depends on accurate signal detection, proper calibration, and reliable communication. The system can be further improved by adding advanced signal processing, better movement recognition, and safety features. Overall, NeuroFlight offers a promising approach to alternative drone control technology.

CHAPTER 6

CONCLUSION & FUTURE SCOPE

6.1 Conclusion

The **NeuroFlight: EMG-Controlled Drone Navigation** is a project that explores controlling a drone using human muscle signals instead of a traditional remote controller. The system uses an EMG sensor to detect muscle activity and an ESP32 microcontroller to process the signals into control commands. These commands are transmitted to the drone to perform the required movements. The project combines sensor technology, embedded systems, and wireless communication to provide an alternative approach to drone navigation.

This project helps demonstrate the potential of human muscle signals in controlling electronic devices and improving human–machine interaction. It also provides practical knowledge of EMG sensors, microcontrollers, and drone control systems. With proper calibration, reliable communication, and suitable safety features, the system can be improved for better performance. Overall, NeuroFlight offers an innovative approach to drone control and provides opportunities for further development in robotics and assistive technology.

6.2 Future Scope

In the future, the **NeuroFlight system** can be improved by using advanced EMG sensors and better signal-processing techniques. These improvements can help detect muscle movements more accurately and reduce errors caused by unwanted signals. The system can also be enhanced to provide faster responses and smoother drone movements. This will help make drone navigation more efficient and reliable.

The project can be further developed by adding more control functions, such as automatic take-off, landing, altitude adjustment, and obstacle detection. Artificial Intelligence (AI) and Machine Learning (ML) techniques can be used to recognize different muscle patterns and convert them into suitable drone commands. GPS and additional sensors can also be integrated to improve navigation and monitoring capabilities.

NeuroFlight has the potential to support future developments in robotics, assistive technology, and human–machine interaction. It may provide an alternative control method for people who find traditional remote controllers difficult to use. In the future, the system could be made more user-friendly, portable, and cost-effective. With further research, testing, and safety improvements, NeuroFlight can become a more advanced and reliable muscle-controlled drone navigation system.

.

References

1. Ali, M. et al. EMG-Based Control of a Quadcopter. 2020.
2. Raez, M. B. I. et al. Techniques of EMG Signal Analysis. BioMedical Engineering Online, 2016.
3. Bitcraze. Crazyflie Drone Programming Documentation.
   Available from: https://www.bitcraze.io/
4. Espressif Systems. ESP-Drone Documentation.
   Available from: https://docs.espressif.com/projects/espressif-esp-drone/
5. MathWorks. Electromyography (EMG) Signal Processing.
   Available from: https://www.mathworks.com/
6. Arduino. Arduino Documentation for Sensor and Controller Integration.
   Available from: https://docs.arduino.cc/
7. PX4 Autopilot. Drone Flight Control Documentation.
   Available from: https://docs.px4.io/
8. ArduPilot. Unmanned Aerial Vehicle Control System.
   Available from: https://ardupilot.org/
9. IEEE. Research on EMG-Based Human–Machine Interaction.
   Available from: https://ieeexplore.ieee.org/
10. ResearchGate. Studies on EMG-Controlled Drone Navigation.
    Available from: https://www.researchgate.net/

POSTER PRESENTATION

MALLA REDDY UNIVERSITY

EMG-Based Drone Control Using NPG Lite and LiteWing ESP32-S3

Towards a Natural and Intuitive Human–Drone Interface

Department of Electrical and Electronics Engineering | University Name

Your Muscles. Our Control. A Smarter Tomorrow.

1. Introduction

This project presents an EMG-based control system for a drone using the NPG Lite biosignal acquisition device and the LiteWing ESP32-S3 drone. The system allows users to control the drone's movement through muscle activity, providing a natural, hands-free and intuitive interface.

Key Objectives

- Acquire and process EMG signals from left and right forearms
- Classify muscle activity into simple commands
- Send commands to the drone via Wi-Fi
- Provide a real-time dashboard for monitoring and control

Natural Control | Hands-Free Operation | Real-Time Response | Low-Cost & Portable

2. System Architecture



3. Hardware & Software

Hardware



NPG Lite Explorer Pack
- NPG Lite (EMG acquisition device)
- PlayGround Lite (development board)



LiteWing ESP32-S3
- Wi-Fi (ESP32-S3)
- 4 brushless motors
- IMU, ToF, optical flow (not detected)

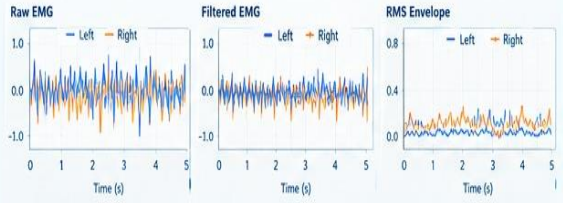
Software

Python 3.11
- Signal processing & classification
- Dashboard (Tkinter)

cflib
- Drone communication
- Motor control

4. EMG Signal Processing

Raw EMG Signal (20–450 Hz) → DC Removal (High-pass filter) → Bandpass Filter (20–100 Hz) → Notch Filter (50 Hz) → RMS Calculation (Feature extraction)



5. Command Classification

Condition	Command
Left EMG activated (Left forearm flexed)	LEFT
Right EMG activated (Right forearm flexed)	RIGHT
Both EMG activated (Both forearms flexed)	STOP
Neither activated (Relaxed / no signal)	IDLE

A configurable classifier (threshold / logic) determines the final command.

6. Live Dashboard



7. Current Implementation Status

Completed
- EMG data recording (CSV)
- Offline signal processing
- Command logic
- Local dashboard (sample data)

In Progress
- NPG Lite (BLE) connection
- LiteWing (Wi-Fi) connection
- Real-time data integration

Pending
- Live command testing
- Motor response verification
- Flight testing

Additional
- Propellers removed during testing
- No flight tests yet
- Safety checks in progress

8. Conclusion

This project demonstrates a functional EMG-based drone control system using NPG Lite and LiteWing ESP32-S3. The integrated pipeline—from signal acquisition to command generation—works reliably and provides a natural, hands-free interface for drone control.

9. Future Work

- Complete BLE and Wi-Fi connectivity tests
- Improve signal filtering and classification accuracy
- Add real-time feedback and user calibration
- Integrate additional sensors (e.g., ToF, optical flow)
- Perform controlled flight tests and safety validation